

Lab8实验报告

小组成员：2311858 许哲涵，2311576 赵宁，2313650 赵悦蛟

一、实验目的

通过完成本次实验，希望能够达到以下目标

- 了解文件系统抽象层-VFS的设计与实现
- 了解基于索引节点组织方式的Simple FS文件系统与操作的设计与实现
- 了解“一切皆为文件”思想的设备文件设计
- 了解简单系统终端的实现

二、实验内容

(一)本实验中重要的知识点

1. 虚拟文件系统 (VFS) 抽象层

- **实验内容**：通过 `struct file`、`struct inode`、`struct fs` 等数据结构抽象不同文件系统的共性操作，提供统一的 `vop_read`、`vop_write` 等操作接口。
- **OS原理**：**虚拟文件系统 (VFS) 抽象层**是操作系统为统一管理不同文件系统而设计的一个中间层。它通过定义一组公共的数据结构（如 `inode`、`file`）和操作接口（如 `open`、`read`），向上为应用程序提供一致的POSIX文件访问API，向下将具体文件系统的实现细节（如SFS、设备文件）封装在私有接口中，实现了“文件系统多样性”与“用户接口一致性”的分离。
- **理解**：VFS是典型的**外观模式**在OS中的应用，它隐藏了SFS、设备文件等不同文件系统的实现细节，为用户提供统一的POSIX接口。实验中的 `vfs_open`、`vfs_lookup` 等函数体现了这一抽象。
- **关系与差异**：原理强调接口统一的设计思想，实验通过具体的数据结构（如 `inode_ops` 函数指针表）实现了这一思想，但实验的VFS比Linux的VFS简化很多。

2. 基于索引节点 (inode) 的文件系统组织

- **实验内容**：SFS采用inode组织文件，磁盘inode包含直接/间接块指针，内存inode缓存元数据。
- **OS原理**：**基于索引节点 (inode) 的文件系统组织**将文件的元数据（权限、大小、时间戳等）与数据存储分离。每个文件对应一个唯一的inode，其中记录数据块在磁盘上的位置（通过直接、间接指针索引）。目录被视为一种特殊的文件，其内容存储了文件名到inode号的映射关系。这种设计使得文件查找、硬链接实现和数据块高效管理成为可能。
- **理解**：inode机制实现了**文件名与文件数据的分离**。实验中的 `sfs_load_inode`、`sfs_bmap_load_nolock` 展示了如何通过inode号找到数据块。
- **关系与差异**：原理描述通用的inode概念，实验实现了具体的SFS inode结构（含12个直接指针和1个间接指针），比现代文件系统简单。

3. “一切皆为文件”的设备抽象

- **实验内容**：设备（如stdin/stdout）也通过文件接口访问，有对应的 `inode_ops`。
- **OS原理**：“**一切皆为文件**”的设备抽象是UNIX哲学的核心体现，它将所有I/O资源（磁盘文件、硬件设备、管道等）统一抽象为文件。设备被赋予文件路径，通过相同的 `open/read/write/ioctl` 接口访问。这简

化了用户程序对异构资源的操作，也为设备驱动提供了标准的框架，使得设备可以像普通文件一样被纳入文件系统的命名空间和管理体系中。

- **理解：**通过VFS将设备映射为文件，实现**统一的read/write接口**。实验中的 `stdin`、`stdout` 作为设备文件处理。
- **关系与差异：**原理涵盖所有设备类型，实验仅实现简单字符设备，且缺少块设备的具体实现。

4.基于文件系统的程序加载与执行

- **实验内容：**修改 `load_icode` 从文件系统（通过fd）而非内存加载ELF，建立进程地址空间。
- **OS原理：基于文件系统的程序加载与执行**是指操作系统从文件系统中读取可执行文件（如ELF格式）到内存并创建进程的过程。系统解析文件头部，为代码段、数据段等分配物理内存并建立页表映射，设置用户栈和初始执行上下文，最后将CPU控制权转移到程序的入口点。这使得程序可以持久存储在磁盘上，按需动态加载，实现了存储与执行的分离。
- **理解：**这是**进程创建的关键步骤**，将存储在文件系统上的可执行代码加载到内存并建立执行环境。实验展示了文件系统与进程管理的交互。
- **关系与差异：**原理包括更复杂的动态链接和共享库，实验仅实现静态ELF加载，且缺少页面的延迟分配等优化。

5.管道机制设计

- **实验内容：**设计管道的数据结构（环形缓冲区、信号量同步）和接口。
- **OS原理：管道机制设计**本质上是进程间通信的一种特殊文件形式。它借助一个内核维护的环形缓冲区，并抽象为两个文件描述符（分别用于读和写）。通过生产者-消费者模型实现同步：当缓冲区满时写进程阻塞，空时读进程阻塞。结合信号量等机制保证数据一致性，管道提供了单向、顺序、字节流的IPC方式，完美融入了“一切皆文件”的抽象体系。
- **理解：**管道是**基于文件的IPC机制**，使用环形缓冲区实现数据传递，通过信号量解决同步问题。实验设计涵盖了数据结构、接口和同步方案。
- **关系与差异：**原理描述管道的行为语义，实验提供了具体实现方案。但实验未实现真正的管道，只是设计方案。

(二)OS原理中很重要，但在实验中没有对应上的知识点

1.磁盘空间分配策略

- **OS原理：磁盘空间分配策略**指的是文件系统如何管理磁盘空闲块，核心是在减少碎片与提高访问速度间权衡。常见策略有位图法（简单但低效）、空闲链表法（灵活但需额外空间）和块组分配法（如ext2将磁盘分块组，提升局部性）。现代系统还会结合预分配和延迟分配等技术优化性能。
- **实验缺失：**SFS使用简单位图分配，没有考虑局部性或碎片整理。

2.文件锁与并发访问控制

- **OS原理：文件锁与并发访问控制**用于协调多进程对同一文件的访问，防止数据混乱。分为建议性锁（依赖进程自觉遵守）和强制性锁（系统强制执行），按粒度又可分为文件级锁和记录级锁。Linux通过 `fcntl()` 系统调用实现，需要处理死锁检测、锁继承等复杂情况，确保读写操作的原子性与一致性。
- **实验缺失：**实验中的文件操作有基本互斥，但没有细粒度锁机制（如区域锁）。

3.内存映射文件（mmap）

- **OS原理重要性：内存映射文件（mmap）**是将文件直接映射到进程虚拟地址空间的技术，实现了文件I/O与内存访问的统一。访问映射区域会触发缺页异常，系统自动加载对应文件页到物理内存，支持写回同

步。mmap不仅避免了用户态与内核态的数据拷贝，还能实现进程间共享内存通信，是高性能文件处理的关键机制。

- **实验缺失**：实验仅支持read/write系统调用，没有实现mmap。

练习0：填写已有实验

要求：本实验依赖实验2/3/4/5/6/7。请把你做的实验2/3/4/5/6/7的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”/“LAB6”/“LAB7”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab8的测试应用程序，可能需对已完成的实验2/3/4/5/6/7的代码进行进一步改进。

练习1：完成读文件操作的实现（需要编码）

要求：首先了解打开文件的处理流程，然后参考本实验后续的文件读写操作的过程分析，填写在kern/fs/sfs/sfs_inode.c中的sfs_io_nolock()函数，实现读文件中数据的代码。

1.打开文件的处理流程：

在用户程序调用 `open("filename", flags)` 发起文件打开请求后，系统首先通过系统调用进入内核态，由 `sysfile_open()` 接收处理。该函数将用户空间的文件路径安全拷贝至内核，并调用 `file_open()` 进入文件系统抽象层。在此层中，系统解析打开标志以确定文件的读写权限，并通过 `fd_array_alloc()` 为当前进程分配一个空闲的文件描述符。随后，核心的打开操作由 `vfs_open()` 执行，包括对打开标志的合法性检查，以及调用 `vfs_lookup()` 根据路径查找文件：其中通过 `get_device()` 解析设备信息和子路径，并根据路径类型获取根文件系统或当前目录的inode。若文件不存在且设置了 `O_CREAT` 标志，则调用 `vop_create()` 创建新文件；找到文件后，通过 `vop_open()` 打开对应的文件节点，并根据需要处理特殊标志——如遇到 `O_TRUNC` 时调用 `vop_truncate()` 清空文件内容，或遇到 `O_APPEND` 时将文件位置指针移至末尾。

接着流程深入至具体的SFS文件系统层，通过 `sfs_lookup()` 解析路径并逐级查找目录项，其中 `sfs_lookup_once()` 和 `sfs_dirent_search_nolock()` 负责在目录中定位目标条目。根据查得的inode编号，`sfs_load_inode()` 将磁盘上的inode信息加载到内存，再经由 `sfs_bmap_load_nolock()` 获取文件数据块的位置信息。

最后，内核创建并初始化 `struct file` 结构，设置文件位置指针与读写权限等属性，并将对应的文件描述符返回给用户程序，从而完成整个打开文件的流程。

2.读文件操作流程

在用户程序调用 `read(fd, buffer, len)` 发起读文件请求后，系统调用将控制权转移至内核，由 `sysfile_read()` 函数处理。

该函数首先检查参数的合法性，并在内核空间分配一个大小为4096字节的缓冲区用于中转数据。读取过程通过一个循环完成，只要未读完指定长度，就会调用 `file_read()` 从文件中读取最多4096字节的数据至内核缓冲区，再通过 `copy_to_user()` 将数据安全地拷贝到用户空间的目标缓冲区，并同步更新已读取的字节数和用户缓冲区位置。

在文件系统抽象层的 `file_read()` 函数中，系统首先通过 `fd2file()` 根据文件描述符获取对应的 `struct file` 结构，并验证该文件是否具有可读权限。接着，它创建一个 `struct iobuf` 结构来描述此次I/O操作的信息，然后调用 `vop_read(file->node, iob)` 进入虚拟文件系统层执行实际的读取操作。

读取完成后，函数会更新文件的位置指针，即 `file->pos` 增加实际读取的字节数。

具体的读取操作**最终由 SFS 文件系统层执行，调用链为** `sfs_read()` -> `sfs_io()` -> `sfs_io_nolock()`。在 `sfs_io_nolock()` 中，系统首先验证请求读取的文件范围是否合法。为了高效处理，实际的读取过程被分为三个逻辑部分：**首先处理起始可能存在的非对齐数据块，调用** `sfs_rbuf()` **进行读取；随后处理中间完全对齐的整块数据，通过** `sfs_rblock()` **进行批量读取以提升效率；最后处理末尾可能存在的非对齐部分，再次调用** `sfs_rbuf()` **读取**。在读取每个部分时，都会通过 `sfs_bmap_load_nolock()` 将文件的逻辑块号映射到磁盘上的物理块号，并最终调用底层的 `ide_read_secs()` 函数从物理磁盘扇区中读取数据，从而完成整个读文件操作。

3.写文件操作流程

写操作与读操作流程类似，主要区别：

(1) 关键不同点

- **权限检查**：在进行权限检查时，系统需要严格验证文件是否具有可写权限，这与读操作的读权限检查形成对应。
- **写函数**：在SFS文件系统层的具体调用函数上，写操作最终调用的是 `sfs_write()` 函数，该函数进一步调用 `sfs_io()` 并传递 `write` 参数为 `true`，从而进入写操作的分支
- **磁盘操作**：写操作不再调用 `sfs_rbuf()` 和 `sfs_rblock()` 进行数据读取，而是调用对应的 `sfs_wbuf()` 和 `sfs_wblock()` 函数，将数据写入到磁盘缓冲区或直接写入磁盘块。
- **文件大小更新**：文件大小的动态管理是写操作独有的重要环节。如果写入操作的起始位置或写入结束位置超过了文件的当前大小，文件系统必须扩展文件。这涉及分配新的数据块、更新文件的inode信息以反映新的文件大小，确保文件能够容纳新写入的数据。
- **脏标志设置**：元数据管理上，由于文件内容或大小被修改，系统需要立即将对应的inode结构体标记为“脏”（即设置 `sin->dirty = 1`），这表示该inode包含的元数据（如文件大小、块位图信息等）已在内存中被修改，后续需要写回磁盘以保证一致性。

(2) 特殊处理

- **文件扩展**：当写入超过当前大小时，需要分配新的数据块
- **延迟写入**：修改可能不会立即写回磁盘，直到调用 `sync()` 或缓冲区满

4.sfs_io_nolock 函数具体代码实现

了解了以上打开文件、读文件和写文件操作的过程，我们想要实现读文件中数据的代码，也就是 `sfs_io_nolock()` 函数，就可以基于块设备的分段处理原理来实现。将读写请求4KB块边界分成三部分：先处理开头不完整的部分，再批量处理中间完整的整块，最后处理结尾不完整的部分。这样既能正确处理任意位置的读写，又能提高完整块的读写效率。

接下来，讲解一下具体代码的实现：

(1) 函数参数和作用

```
static int sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, void *buf,
                        off_t offset, size_t *alenp, bool write)
```

- `sfs`：这是指向整个Simple File System文件系统控制结构的指针，它包含了文件系统的全局信息，比如超级块数据、空闲块管理信息等，通过这个指针可以访问整个文件系统的元数据和操作函数。
- `sin`：这是内存中某个文件的索引节点结构指针，包含了该文件在SFS中的具体信息，比如指向磁盘inode的指针、文件大小、数据块位置等。每个打开的文件在内存中都有一个对应的sin结构，用于缓存文件的元数据信息。

- `buf`: 这是内核空间中用于读写数据的缓冲区地址。对于读操作，从磁盘读取的数据会存放到这个缓冲区；对于写操作，要写入磁盘的数据从这个缓冲区取出。注意这是内核地址，不是用户空间地址。
- `offset`: 这是相对于文件起始位置的字节偏移量，表示要从文件的哪个位置开始读写。比如`offset=1000`表示从文件第1000字节处开始操作。
- `alenp`: 这是一个输入输出参数。调用函数时，`alenp`表示请求读写的数据长度；函数返回时，`alenp`会被修改为实际成功读写的字节数。比如请求读1000字节但文件只有500字节，那么实际只会读500字节。
- `write`: 这是一个布尔标志，`true`表示执行写操作（将`buf`中的数据写入文件），`false`表示执行读操作（从文件读取数据到`buf`中）。

(2) 预备处理

1) 边界检查:

```
off_t endpos = offset + *alenp, blkoff;
*alenp = 0; // 先清零，最后再设置实际值
```

- `off_t endpos = offset + *alenp, blkoff;` 这行代码计算请求的结束位置，就是起始偏移量加上请求长度，得到要读写的最后一个字节之后的位置。
- `*alenp = 0;` 先把实际读写长度清零，是因为后面会分段计算实际完成的读写量，最后再设置这个值。

这样检查的核心目的是：**保证所有读写操作都在文件系统允许的范围内，并且对于读操作不能读取未分配的空间，对于写操作可以自动扩展文件大小但也不能超过文件系统限制。**

2) 参数合法性验证

```
// 偏移量不能为负，不能超过文件最大大小
if (offset < 0 || offset >= SFS_MAX_FILE_SIZE || offset > endpos) {
    return -E_INVALID;
}
// 读操作的特殊检查：不能超过文件当前大小
if (!write) {
    if (offset >= din->size) {
        return 0; // 读的位置已经超过文件末尾
    }
    if (endpos > din->size) {
        endpos = din->size; // 调整结束位置到文件末尾
    }
}
```

这个参数合法性验证是为了确保文件读写操作的安全性和正确性。简单来说就是防止程序读写不该访问的内存区域：偏移量不能是负数（因为文件位置从0开始），也不能超过系统支持的最大文件限制，还要确保起始位置不大于结束位置（防止长度计算错误）。对于读操作有更严格的限制——不能读取文件实际大小之外的数据，如果读取起点已经超过文件末尾就直接返回空，如果要读取的范围超出了文件大小就把结束位置调整到文件末尾。这样既能防止访问未分配的空间导致错误，又能正确处理文件边界情况，确保读写操作既安全又符合预期。

3) 设置操作函数指针


```

int (*sfs_buf_op)(struct sfs_fs *sfs, void *buf, size_t len, uint32_t blkno, off_t offset);
int (*sfs_block_op)(struct sfs_fs *sfs, void *buf, uint32_t blkno, uint32_t nblks);

if (write) {
    sfs_buf_op = sfs_wbuf, sfs_block_op = sfs_wblock; // 写操作
} else {
    sfs_buf_op = sfs_rbuf, sfs_block_op = sfs_rblock; // 读操作
}

```

这段代码是设置读写操作的具体函数指针，为后面执行实际的磁盘操作做准备。

它定义了两个函数指针变量：`sfs_buf_op` 用于处理块内的部分数据读写，比如一个块的开头或结尾不完整部分，`sfs_block_op` 用于处理整块数据的批量读写。然后根据 `write` 参数的值来决定具体指向哪个函数——如果是写操作就指向 `sfs_wbuf` 和 `sfs_wblock` 这两个写函数，如果是读操作就指向 `sfs_rbuf` 和 `sfs_rblock` 这两个读函数。这样设计的好处是后面的代码可以用统一的接口调用不同的读写实现，不需要在每个读写分支都写重复的逻辑。

(3) 核心的三段式读写处理

SFS 将读写请求分成三部分处理：

1) 处理起始未对齐部分

```

// 检查偏移量是否块对齐
if ((blkoff = offset % SFS_BLKSIZE) != 0) {
    // 计算要读写的长度：如果有后续块，就读完当前块；否则只读到请求结束
    size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);

    // 获取数据块的物理块号
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

    // 执行读写操作（sfs_buf_op 可能是 sfs_rbuf 或 sfs_wbuf）
    if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
        goto out;
    }

    // 更新状态
    alen += size, buf += size, blkno ++;
    if (nblks > 0) {
        nblks --; // 消耗了一个块
    }
}

```

这段代码处理文件读写请求中开头不完整的部分。

它的作用是：当读写起始位置不在4KB块边界上时（比如从第1000字节开始，而块边界是4096字节），先处理这个开头的不完整片段。算法是这样的：先计算块内偏移量 `blkoff`（比如1000），然后判断——如果后面还有完整的块要处理，就一次读完当前块的剩余部分（4096-1000=3096字节）；如果后面没有完整块了，就直接读到请求结束的位置。接着通过 `sfs_bmap_load_nolock` 找到这个块在磁盘上的实际位置，用 `sfs_buf_op` 执行实际读写，最后更新已经处理的字节数、缓冲区指针和块计数器，为后面的处理做好准备。这样确保了即使读写不从块边界开始，也能正确读取第一块内的部分数据。

2) 处理中间对齐的完整块

```
size = SFS_BLKSIZE;
while (nblks > 0) {
    // 获取每个数据块的物理块号
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

    // 执行块读写操作（更高效，一次读写整个块）
    if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
        goto out;
    }

    // 更新状态
    alen += size, buf += size, blkno ++, nblks --;
}
```

这段代码负责处理读写请求中间那些完整的4KB数据块。

它的作用是高效地批量处理对齐的整块数据：通过while循环依次处理每一个完整块，每次循环先用 `sfs_bmap_load_nolock` 找到当前逻辑块对应的物理磁盘块号，然后调用 `sfs_block_op` 一次性读写整个块（4KB），这样比多次读写部分数据更高效。每次处理完一个块就更新已处理字节数、缓冲区指针位置、当前块号和剩余块数，直到所有完整块都处理完毕。这样可以最大化磁盘读写效率，减少系统调用和磁盘寻址的开销。

3) 处理末尾未对齐部分

```
// 计算剩余未处理的字节数
if ((size = endpos - offset - alen) > 0) {
    // 获取最后一个块的物理块号
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

    // 从块的起始位置（偏移0）开始读写剩余数据
    if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
        goto out;
    }

    alen += size;
}
```

这段代码处理读写请求末尾的不完整部分。

它的作用是：在已经处理完开头片段和中间完整块之后，计算还剩下多少字节没处理，用请求的总结束位置减去起始位置再减去已处理长度，如果还有剩余数据就处理最后这个不完整的块。具体实现是先用 `sfs_bmap_load_nolock` 找到最后一个块的物理位置，然后从该块的起始位置也就是偏移0开始，用 `sfs_buf_op` 读写剩余的数据量，最后更新总处理字节数。这样确保整个读写请求从头到尾的所有数据都被正确处理，即使是结尾不在块边界上的零碎数据也能完整处理。

练习2: 完成基于文件系统的执行程序机制的实现（需要编码）

要求：改写proc.c中的load_icode函数和其他相关函数，实现基于文件系统的执行程序机制。执行：make qemu。如果能看到sh用户程序的执行界面，则基本成功了。如果在sh用户界面上可以执行 `exit`, `hello`（更多用户程序放在 `user` 目录下）等其他放置在 `sfs` 文件系统系统中的其他执行程序，则可以认为本实验基本成功。

我们想要完成基于文件系统的执行程序机制，就需要实现**将ELF格式的可执行文件从磁盘读入内存，建立用户进程的虚拟地址空间，并设置正确的执行环境。**

具体来说，要通过文件描述符读取ELF文件头，解析出代码段、数据段等信息，为每个段分配物理内存并建立页表映射，然后将文件中的程序代码和数据复制到分配的内存中，最后设置用户栈、命令行参数和陷阱帧，让CPU切换到用户态执行程序的入口点，从而实现基于文件系统的程序加载和执行。

原来的 `load_icode` 函数是从内存中的二进制数据加载程序，现在改为通过**文件描述符fd**从文件系统读取程序文件。具体实现步骤如下：

(1) 获取ELF文件头

```
// 从文件读取ELF头
if ((ret = load_icode_read(fd, elf, sizeof(struct elfhdr), 0)) != 0)
```

通过文件系统接口 `load_icode_read` 从文件读取ELF头部信息，验证魔数 `ELF_MAGIC` 确保是合法的ELF文件。

(2) 解析程序头表

```
// 遍历所有程序头
for (int i = 0; i < elf->e_phnum; i++) {
    // 从文件中读取每个程序头
    if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr), phoff)) != 0)
}
```

从文件中读取每个程序头，只处理 `ELF_PT_LOAD` 类型的段，也就是可加载段。

(3) 建立内存映射

```
// 为TEXT/DATA段建立虚拟内存区域
if ((ret = mm_map(mm, ph->p_va, ph->p_memsz, vm_flags, NULL)) != 0)
```

为每个段建立VMA（虚拟内存区域），设置正确的权限，比如可读、可写、可执行。

(4) 加载文件内容

```
// 从文件读取数据到分配的内存页面
if ((ret = load_icode_read(fd, kva + off, size, offset)) != 0)
```

关键改动：**从文件读取数据**而不是从内存中的二进制数组复制。

- `kva`：内核虚拟地址，指向分配的用户页面
- `offset`：在ELF文件中的偏移量
- `size`：要读取的字节数

(5) 处理BSS段


```
// BSS段（未初始化数据）清零
memset(kva + off, 0, size);
```

BSS段在文件中不占用空间，但在内存中需要分配并清零。

以上实现，最终实现效果如下所示：

```

yield.c create bin/sfs.img (disk0) successfully. + cc kern/init/entry.S + cc ker
n/init/init.c + cc kern/libs/readline.c + cc kern/libs/stdio.c + cc kern/libs/st
ring.c + cc kern/debug/kdebug.c + cc kern/debug/kmonitor.c + cc kern/debug/panic
.c + cc kern/driver/clock.c + cc kern/driver/console.c + cc kern/driver/dtb.c +
cc kern/driver/ide.c + cc kern/driver/intr.c + cc kern/driver/picirq.c + cc kern
/driver/ramdisk.c + cc kern/trap/trap.c + cc kern/trap/trapentry.S + cc kern/mm/
default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc
kern/sync/check_sync.c + cc kern/sync/monitor.c + cc kern/sync/sem.c + cc kern/s
ync/wait.c + cc kern/fs/file.c + cc kern/fs/fs.c + cc kern/fs/iobuf.c + cc kern/
fs/sysfile.c + cc kern/process/entry.S + cc kern/process/proc.c + cc kern/proces
s/switch.S + cc kern/schedule/default_sched.c + cc kern/schedule/default_sched_s
tride.c + cc kern/schedule/sched.c + cc kern/syscall/syscall.c + cc kern/fs/swap
/swapfs.c + cc kern/fs/vfs/inode.c + cc kern/fs/vfs/vfs.c + cc kern/fs/vfs/vfsde
v.c + cc kern/fs/vfs/vfsfile.c + cc kern/fs/vfs/vfslookup.c + cc kern/fs/vfs/vfs
path.c + cc kern/fs/devs/dev.c + cc kern/fs/devs/dev_disk0.c + cc kern/fs/devs/d
ev_stdin.c + cc kern/fs/devs/dev_stdout.c + cc kern/fs/sfs/bitmap.c + cc kern/fs
/sfs/sfs.c + cc kern/fs/sfs/sfs_fs.c + cc kern/fs/sfs/sfs_inode.c + cc kern/fs/s
fs/sfs_io.c + cc kern/fs/sfs/sfs_lock.c + ld bin/kernel riscv64-unknown-elf-objc
opy bin/kernel --strip-all -O binary bin/ucore.img gmake[1]: 离开目录"/home/ubun
tu/labcode/lab8_final"
-sh execve: OK
-user sh : OK
Total Score: 100/100
ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab8_final$

```

实现了以上功能后，发现想要实现在sh用户界面上可以执行 `exit`，`hello` 等用户程序，还要进行一些修改。一开始用户程序 `sh` 虽能正常启动，但在 `$` 提示符后无法接收输入，每次按键都会触发 "Store/AMO page fault" 错误。

经过分析，问题根源在于操作系统缺乏完整的页错误处理机制：当用户程序访问未映射的虚拟地址时，系统仅打印错误信息而未实际分配物理页面。为此，我们进行了三处关键修改。

首先，在 `vmm.c` 中添加了 `do_pgfault` 函数，该函数实现了完整的页错误处理流程，包括地址验证、虚拟内存区域查找、栈的动态扩展、权限检查及物理页面分配。具体函数如下：

```
int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr) {
    int ret = -E_INVALID;

    // 检查地址是否在用户空间范围内
    if (!USER_ACCESS(addr, addr + 1)) {
        printf("do_pgfault: invalid address %08x\n", addr);
        goto failed;
    }
}
```

```

}

// 查找包含此地址的vma
struct vma_struct *vma = find_vma(mm, addr);

// 如果没有找到vma, 检查是否是栈扩展
if (vma == NULL || vma->vm_start > addr) {
    // 检查是否是栈区域的地址 - 使用memlayout.h中的定义
    if (addr >= USTACKTOP - USTACKSIZE && addr < USTACKTOP) {
        // 栈扩展, 创建新的栈vma或扩展现有栈vma
        uintptr_t stack_start = ROUNDDOWN(addr, PGSIZE);
        uintptr_t stack_end = ROUNDUP(addr + 1, PGSIZE);

        // 查找栈vma
        vma = find_vma(mm, USTACKTOP - 1);
        if (vma != NULL && (vma->vm_flags & VM_STACK)) {
            // 扩展现有栈vma
            if (stack_start < vma->vm_start) {
                vma->vm_start = stack_start;
            }
        } else {
            // 创建新的栈vma
            vma = vma_create(stack_start, stack_end, VM_READ | VM_WRITE | VM_STACK);
            if (vma == NULL) {
                cprintf("do_pgfault: failed to create stack vma\n");
                goto failed;
            }
            insert_vma_struct(mm, vma);
        }
    } else if (addr >= UTEXT && addr < USTACKTOP - USTACKSIZE) {
        // 可能是堆区域或数据区域, 在UTEXT到栈之间
        // 创建临时的vma
        vma = vma_create(ROUNDDOWN(addr, PGSIZE),
                        ROUNDUP(addr + 1, PGSIZE),
                        VM_READ | VM_WRITE);
        if (vma == NULL) {
            cprintf("do_pgfault: failed to create heap/data vma\n");
            goto failed;
        }
        insert_vma_struct(mm, vma);
    } else {
        cprintf("do_pgfault: no vma contains addr %08x\n", addr);
        goto failed;
    }
}

// 检查错误类型和vma权限
uint32_t perm = PTE_U;
if (error_code == CAUSE_STORE_PAGE_FAULT) {
    // 写错误, 需要写权限
    if (!(vma->vm_flags & VM_WRITE)) {
        cprintf("do_pgfault: write to non-writable vma, addr=%08x, flags=%x\n",
            addr, vma->vm_flags);
    }
}

```

```

        goto failed;
    }
    perm |= PTE_W | PTE_R; // RISC-V中，可写页必须可读
} else if (error_code == CAUSE_LOAD_PAGE_FAULT) {
    // 读错误，需要读权限
    if (!(vma->vm_flags & VM_READ)) {
        cprintf("do_pgfault: read from non-readable vma\n");
        goto failed;
    }
    perm |= PTE_R;
} else if (error_code == CAUSE_FETCH_PAGE_FAULT) {
    // 执行错误，需要执行权限
    if (!(vma->vm_flags & VM_EXEC)) {
        cprintf("do_pgfault: execute from non-executable vma\n");
        goto failed;
    }
    perm |= PTE_X;
}

// 确保页表权限正确
if (vma->vm_flags & VM_READ) perm |= PTE_R;
if (vma->vm_flags & VM_WRITE) perm |= (PTE_W | PTE_R);
if (vma->vm_flags & VM_EXEC) perm |= PTE_X;

// 调用pgdir_alloc_page分配页面
struct Page *page = pgdir_alloc_page(mm->pgdir, addr, perm);
if (page == NULL) {
    cprintf("do_pgfault: pgdir_alloc_page failed for addr %08x\n", addr);
    ret = -E_NO_MEM;
    goto failed;
}

// 清空页面内容
memset(page2kva(page), 0, PGSIZE);

return 0;

failed:
    return ret;
}

```

其次，为确保使用正确的地址空间布局常量，在 `vmm.c` 开头添加了 `#include <memlayout.h>`。

最后，修改了 `trap.c` 中的 `exception_handler` 函数，将页错误异常统一交由 `do_pgfault` 处理，并完善了错误处理逻辑。

```

void exception_handler(struct trapframe *tf)
{
    int ret;
    switch (tf->cause)
    {
        case CAUSE_MISALIGNED_FETCH:
            cprintf("Instruction address misaligned\n");

```

```

        break;
    case CAUSE_FETCH_ACCESS:
        cprintf("Instruction access fault\n");
        break;
    case CAUSE_ILLEGAL_INSTRUCTION:
        cprintf("Illegal instruction\n");
        break;
    case CAUSE_BREAKPOINT:
        cprintf("Breakpoint\n");
        break;
    case CAUSE_MISALIGNED_LOAD:
        cprintf("Load address misaligned\n");
        break;
    case CAUSE_LOAD_ACCESS:
        cprintf("Load access fault\n");
        break;
    case CAUSE_MISALIGNED_STORE:
        panic("AMO address misaligned\n");
        break;
    case CAUSE_STORE_ACCESS:
        cprintf("Store/AMO access fault\n");
        break;
    case CAUSE_USER_ECALL:
        // cprintf("Environment call from U-mode\n");
        tf->epc += 4;
        syscall();
        break;
    case CAUSE_SUPERVISOR_ECALL:
        cprintf("Environment call from S-mode\n");
        tf->epc += 4;
        syscall();
        break;
    case CAUSE_HYPERVISOR_ECALL:
        cprintf("Environment call from H-mode\n");
        break;
    case CAUSE_MACHINE_ECALL:
        cprintf("Environment call from M-mode\n");
        break;
    case CAUSE_FETCH_PAGE_FAULT:
    case CAUSE_LOAD_PAGE_FAULT:
    case CAUSE_STORE_PAGE_FAULT:
        if (current != NULL && current->mm != NULL) {
            ret = do_pgfault(current->mm, tf->cause, tf->tval); //修改之处，调用do_pgfault 函数
// 用户态页错误处理失败，终止进程
            if (ret != 0) {
                cprintf("do_pgfault failed: %e at 0x%08x\n", ret, tf->tval);
                print_trapframe(tf);
                if (trap_in_kernel(tf)) {
                    panic("page fault in kernel mode!\n");
                }
                do_exit(-E_KILLED);
            }
        }
    }
}

```

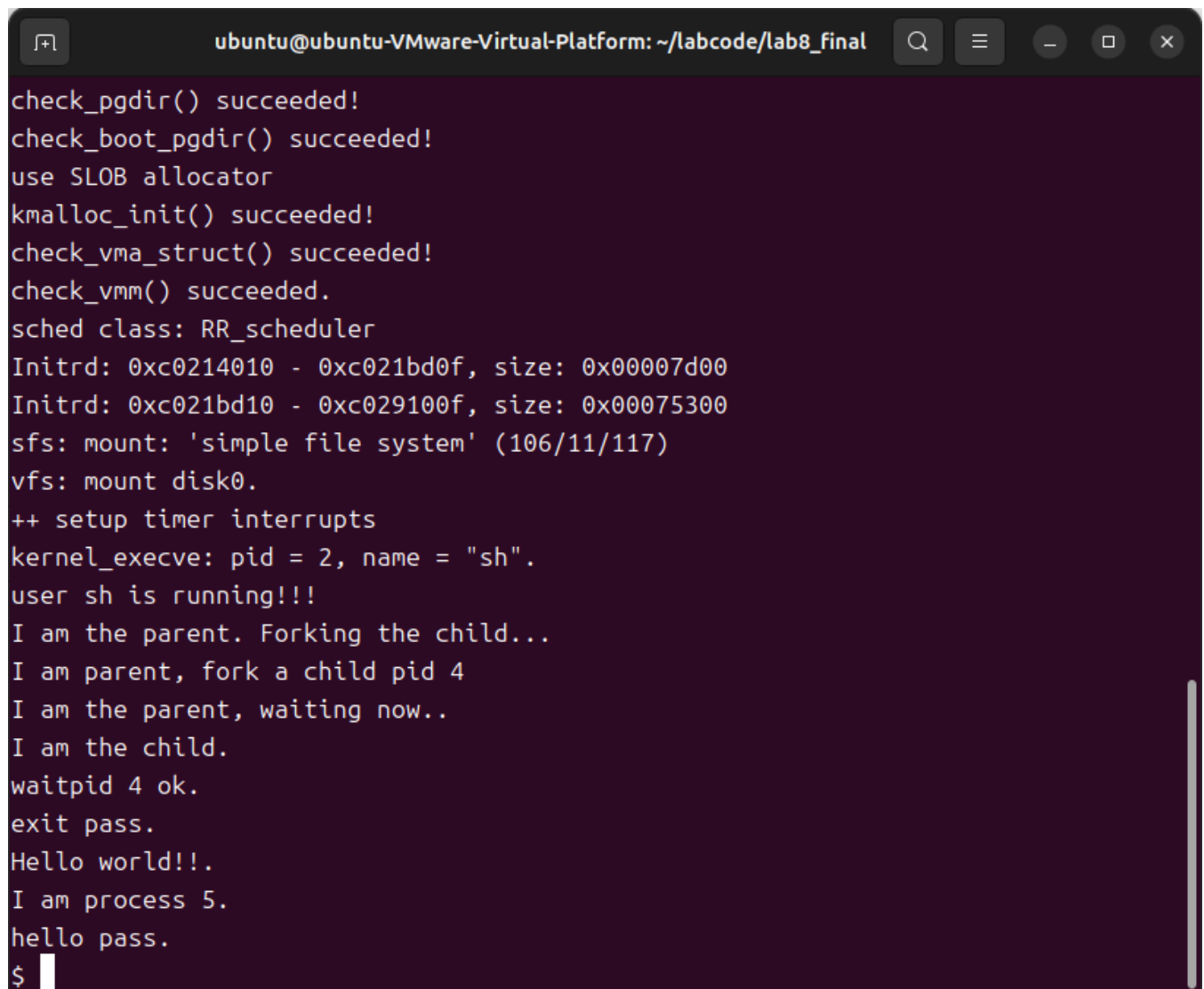
进行处理

```

    } else {
        print_trapframe(tf);
        panic("unhandled page fault at 0x%08x\n", tf->tval);
    }
    break;
default:
    print_trapframe(tf);
    break;
}
}
}

```

然后就能实现在sh用户界面上可以执行 `exit`, `hello` 等用户程序了，经过测试所有的用户程序都能实现，在此我们只展示了hello用户程序：



```

ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab8_final
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
I am the parent. Forking the child...
I am parent, fork a child pid 4
I am the parent, waiting now..
I am the child.
waitpid 4 ok.
exit pass.
Hello world!..
I am process 5.
hello pass.
$

```

扩展练习 Challenge1：完成基于“UNIX的PIPE机制”的设计方案

要求：如果要在ucore里加入UNIX的管道（Pipe）机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个（或多个）具体的C语言struct定义。在网络上查找相关的Linux资料和实现，请在实验报告中给出设计实现“UNIX的PIPE机制”的概要设方案，你的设计应当体现出对可能出现的同步互斥问题的处理。）

管道实现的核心原理是在内存中**创建一个环形缓冲区作为数据中转站，通过两个位置指针实现循环读写。**

它的巧妙之处在于**用信号量解决了读写同步问题**：**写数据前要申请空槽位，确保有空间才写，写完通知有数据可读；读数据前要等待有数据，读完通知有空槽。**

这样既**防止了缓冲区溢出，又避免了读空操作**。多个进程可以**共享同一个管道**，通过引用计数管理生命周期，所有写端都关闭后读端才会收到结束信号。虽然管道数据只在内存中流转，但它通过虚拟文件系统的统一接口暴露给用户，让进程可以用普通的文件读写操作进行通信，实现了高效的进程间数据传递。

1.核心数据结构设计

管道本质上是一种特殊的文件，用于在两个进程间传递数据。在ucore中，我们需要设计以下数据结构：

```
// 管道数据结构
struct pipe {
    struct inode *inode;           // 关联的inode
    char *buffer;                 // 环形缓冲区
    uint32_t size;                // 缓冲区大小（默认4096字节）
    uint32_t read_pos;            // 读位置
    uint32_t write_pos;           // 写位置
    uint32_t data_count;          // 当前缓冲区中的数据量
    semaphore_t sem_empty;        // 空槽信号量（用于同步）
    semaphore_t sem_full;         // 满槽信号量（用于同步）
    semaphore_t mutex;            // 互斥锁，保护缓冲区操作
    int reader_count;              // 读端引用计数
    int writer_count;             // 写端引用计数
    bool reader_closed;           // 读端是否关闭
    bool writer_closed;           // 写端是否关闭
    wait_queue_t read_wait;       // 读等待队列
    wait_queue_t write_wait;      // 写等待队列
};

// inode中增加管道类型支持
struct inode {
    union {                       // 包含不同文件系统特定inode信息的union成员变量
        struct device __device_info; // 设备文件系统内存inode信息
        struct sfs_inode __sfs_inode_info; // SFS文件系统内存inode信息
        struct pipe __pipe_info; // 新增：管道信息
    } in_info;
    enum {
        inode_type_device_info = 0x1234,
        inode_type_sfs_inode_info,
        inode_type_pipe_info, // 新增：管道类型
    } in_type; // 此inode所属文件系统类型
    atomic_t ref_count; // 此inode的引用计数
    atomic_t open_count; // 打开此inode对应文件的个数
    struct fs *in_fs; // 抽象的文件系统，包含访问文件系统的函数指针
    const struct inode_ops *in_ops; // 抽象的inode操作，包含访问inode的函数指针
};
```

这个结构体这样设计是因为**管道虽然通过文件系统接口访问，但本质上是内存中的通信机制而非真正的磁盘文件**。struct pipe中的buffer是环形缓冲区，用来临时存储进程间传递的数据；read_pos和write_pos这两个位置指针让缓冲区能循环使用；sem_empty和sem_full这两个信号量解决了“什么时候能写”和“什么时候能读”的同步问题，防止缓冲区溢出或读空；mutex保护实际的读写操作不会冲突。reader_count和writer_count记录有多少进程打开了管

道的两端，确保所有相关进程都关闭后管道才被销毁。wait_queue让进程在没有数据可读或没有空间可写时能睡眠等待而不是忙等。整个结构体被封装在inode的union里，这样管道就能以普通文件的形式通过VFS接口访问，既保持了“一切皆文件”的统一性，又实现了高效的进程间通信。

2.关键接口设计

(1) 管道创建接口

```
// 创建管道，返回两个文件描述符：fd[0]用于读，fd[1]用于写
int pipe_create(int fd[2]);
```

这个int pipe_create(int fd[2])函数创建一个管道，它接受一个长度为2的整数数组作为参数，调用成功后会把两个文件描述符填到这个数组里。**第一个文件描述符fd[0]是管道的读端，用来从管道读取数据；第二个文件描述符fd[1]是管道的写端，用来向管道写入数据。**

这样设计的原因是管道本质上是一个单向通信通道，一端只读一端只写，操作系统需要给用户两个不同的“把手”来操作这两端。函数返回0表示创建成功，返回负数表示失败。这种设计是UNIX系统的传统接口，保持了与标准管道API的一致性。

(2) 管道读写接口

```
// 管道读操作
int pipe_read(struct pipe *p, void *buf, size_t count);
// 管道写操作
int pipe_write(struct pipe *p, const void *buf, size_t count);
```

这两个函数是管道操作的核心实现。pipe_read函数从管道结构体p指向的环形缓冲区里读取最多count字节的数据，存放到buf指向的内存中，返回实际读取的字节数。pipe_write函数把buf指向的count字节数据写入管道缓冲区，返回实际写入的字节数。它们内部会处理所有的同步问题：**读的时候如果缓冲区没数据就等待，有数据就读取并通知写端有了空位；写的时候如果缓冲区没空间就等待，有空间就写入并通知读端有了新数据。如果管道的一端已经关闭，这些函数会相应地返回0（读端发现写端关闭）或触发错误（写端发现读端关闭）。**

(3) 管道关闭接口

```
// 关闭管道的一端
int pipe_close(struct pipe *p, int mode); // mode: PIPE_READ_END / PIPE_WRITE_END
```

这个pipe_close函数用于关闭管道的某一端，它需要两个参数：管道结构体指针p和一个模式参数mode（用PIPE_READ_END表示关闭读端，PIPE_WRITE_END表示关闭写端）。

函数的作用是**递减相应端的引用计数**，如果引用计数减到0就真正关闭那端，设置reader_closed或writer_closed标志位，并且唤醒可能在另一端等待的进程。比如关闭写端时会唤醒所有在read_wait队列中等待数据的读进程，让它们知道不会再有新数据了；关闭读端时会唤醒在write_wait队列中等待空间的写进程，让它们知道读端已经关闭了（这通常会导致写进程收到SIGPIPE信号）。这个设计保证了管道能正确清理资源并通知相关进程状态变化。

3.同步互斥处理方案

管道机制面临的**核心并发问题是生产者和消费者问题**，生产者和消费者问题好比一个食堂里厨师做饭和学生吃饭的关系。厨师是生产者，不停地做好饭菜放到餐台上；学生是消费者，从餐台上取饭菜吃。这里就会出现几个矛盾：如果厨师做得太快，餐台摆满了没地方放新菜，厨师就得等着；如果学生来得太慢，餐台上的菜都凉了还没人吃，造成了浪费；更麻烦的是如果厨师正在往某个位置放菜，同时有学生伸手去拿同一个位置的菜，那菜可能被打翻。

管道机制面临的正是这个经典问题，不过场景换成了进程间的数据传递。写进程是生产者，往管道的缓冲区里放数据；读进程是消费者，从缓冲区里取数据。读写同步的核心难题在于如何让生产者和消费者协调工作，既不让生产者做无用功（写了数据没人读导致缓冲区满），也不让消费者干等着（想读数据但缓冲区空），还要防止生产者和消费者同时操作把数据弄乱。

为了解决这三个问题，管道采用了**三信号量方案**：

(1) 读写同步

- **空槽信号量 (sem_empty)**：初始值为缓冲区大小，表示可写入的空槽数。它就像餐台的空位计数器，厨师放菜前要看这个计数器，如果是0说明没空位了就得等着，学生拿走一道菜后这个计数器就加一。
- **满槽信号量 (sem_full)**：初始值为0，表示已填充的槽数。它就像是餐台上的菜盘计数器，学生取菜前要看这个计数器，如果是0说明没菜了就得等着，厨师放上一道菜后这个计数器就加一。
- **互斥锁 (mutex)**：保护缓冲区数据结构的完整性。就像是餐台的操作权牌子，无论是厨师放菜还是学生取菜，都要先拿到这个牌子才能操作餐台，操作完再挂回去，这样保证了同一时间只有一个人在操作餐台，不会出现厨师和学生同时伸手的混乱场面。

通过这三个信号量的配合，管道实现了完美的读写同步：写进程在没有空位时自动休眠，有空位时才写入；读进程在没有数据时自动休眠，有数据时才读取；读写操作本身互不干扰。这样就形成了一个高效、安全的数据传递通道，既不会丢数据，也不会产生冲突。

(2) 读写流程

写操作流程：

```
down(&p->sem_empty);    // 等待有空槽
down(&p->mutex);          // 获取缓冲区锁
// 写入数据到缓冲区
up(&p->mutex);            // 释放缓冲区锁
up(&p->sem_full);         // 通知有数据可读
```

这段代码展示了管道**写操作的核心流程**，就像厨师往餐台上放菜的过程。`down(&p->sem_empty)`是厨师先看看餐台上还有没有空位置，如果没有空位（信号量值为0），厨师就在这里等着，直到有学生取走菜腾出空位。等有空位了，厨师执行 `down(&p->mutex)`，这相当于厨师拿起“操作中”的牌子挂在餐台上，告诉其他人“我现在要放菜了，请稍等”。然后厨师把菜放到餐台的指定位置，这就是写入数据到缓冲区。放完菜后，厨师执行 `up(&p->mutex)`，把“操作中”牌子取下来，表示操作完成了，其他人可以来操作了。最后厨师执行 `up(&p->sem_full)`，这相当于把“有菜数量”的计数器加一，通知学生们“有新菜上桌了，可以来吃了”。这个顺序很重要，**必须先确认有空位再拿操作权，操作完先还操作权再通知有菜，这样才能保证整个系统协调有序，不会出现死锁或者数据混乱的情况。**

读操作流程：

```

down(&p->sem_full);           // 等待有数据
down(&p->mutex);               // 获取缓冲区锁
// 从缓冲区读取数据
up(&p->mutex);                 // 释放缓冲区锁
up(&p->sem_empty);             // 通知有空槽

```

这段代码是**管道读操作的核心流程**，就像学生从餐台上取菜吃饭的过程。`down(&p->sem_full)` 是学生先看看餐台上有没有菜可以吃，如果餐台空荡荡的（信号量为0），学生就在这里等着，直到厨师做好新菜放上来。等有菜了，学生执行 `down(&p->mutex)`，这相当于学生拿起“操作中”的牌子挂在餐台上，告诉厨师和其他学生“我现在要取菜了，请稍等”。然后学生从餐台的指定位置取走菜，这就是从缓冲区读取数据。取完菜后，学生执行 `up(&p->mutex)`，把“操作中”牌子取下来，表示操作完成了，其他人可以来操作了。最后学生执行 `up(&p->sem_empty)`，这相当于把“空位数量”的计数器加一，通知厨师“我取走了一道菜，腾出空位了，可以做新菜了”。**这个流程和写操作是对称的，一个管数据的生产，一个管数据的消费，通过信号量的增减实现了完美的配合。**

(3) 边界情况处理

- **所有读端关闭**：继续写入会触发SIGPIPE信号。假设管道有两端，A端写数据，B端读数据。如果B端关闭了（进程退出或者主动关闭了读端），但A端还在继续往管道里写数据，这时候A端的写操作就会失败。在UNIX系统中，操作系统会给A进程发送一个SIGPIPE信号（管道破裂信号），通常这会导致进程终止。在ucore中，由于信号机制可能不完善，可以让 `pipe_write` 函数直接返回一个错误码，比如 `-EPIPE`，告诉写进程“别写了，对面没人听了”。
- **所有写端关闭**：读操作读到EOF。如果管道的所有写端都关闭了，读端还在尝试读取数据，它会先读完缓冲区里剩余的所有数据，然后下一次读操作就会返回0字节，在UNIX中read返回0表示文件结束。
- **缓冲区满**：写操作阻塞，直到有空间。如果写进程还想继续写入数据，它就会在 `down(&sem_empty)` 这一步阻塞住，因为 `sem_empty` 信号量已经减到0了。写进程会一直睡眠等待，直到某个读进程取走一些数据，执行 `up(&sem_empty)` 增加了空槽数量，写进程才会被唤醒继续工作。
- **缓冲区空**：读操作阻塞，直到有数据。读进程想要读取数据，但管道缓冲区里什么都没有，这时候它会在 `down(&sem_full)` 这一步阻塞，因为 `sem_full` 信号量为0。读进程会一直等待，直到某个写进程写入数据后执行 `up(&sem_full)`，读进程才会被唤醒开始读取。

4.与文件系统的集成

管道虽然是特殊文件，但**应当与普通文件使用相同的VFS接口**：

```

// 管道的inode操作
static const struct inode_ops pipe_node_ops = {
    .vop_magic = VOP_MAGIC,
    .vop_open = pipe_open,
    .vop_close = pipe_close,
    .vop_read = pipe_read,
    .vop_write = pipe_write,
    .vop_fstat = pipe_fstat,
    .vop_gettype = pipe_gettype,
    .vop_lookup = pipe_lookup, // 管道不支持查找
};

```

这段代码定义了管道这种特殊文件类型的操作函数表，它的作用是让管道能够通过虚拟文件系统的统一接口被访问。就像给管道这个“特殊演员”配了一套标准的“表演剧本”，当用户程序对管道文件调用open、read、write这些标准文件操作时，VFS会查到这个pipe_node_ops表，然后调用对应的pipe_open、pipe_read、pipe_write等具体实现函数。这样管道虽然内部机制和普通磁盘文件完全不同（数据在内存缓冲区里流转而不是磁盘上），但从用户角度看和操作普通文件一模一样，都是通过那几个标准系统调用来读写，实现了UNIX“一切皆文件”的设计哲学。那个pipe_lookup函数被设为不支持查找是因为管道作为一种临时通信机制，本身不包含子目录或文件，自然也就不支持路径查找操作。

扩展练习 Challenge2：完成基于“UNIX的软连接和硬连接机制”的设计方案

要求：如果要在ucore里加入UNIX的软连接和硬连接机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个（或多个）具体的C语言struct定义。在网络上查找相关的Linux资料和实现，请在实验报告中给出设计实现“UNIX的软连接和硬连接机制”的概要设方案，你的设计应当体现出对可能出现的同步互斥问题的处理。）

这个Challenge要你设计的是UNIX系统中两种不同类型的“文件快捷方式”机制。

可以想象为我们的书房里有一本很厚的《操作系统原理》教科书（这就是原始文件）。**硬链接**就像我在书架上为这本书贴了两个标签，一个写着“OS教材”，另一个写着“考研参考书”，两个标签都指向同一本实体书。无论我用哪个标签找到这本书，看的都是同一本实体书。如果我把“OS教材”这个标签撕掉，书还在那里，“考研参考书”标签依然有效。只有把两个标签都撕掉，这本书才会被真正清理掉。

软链接（符号链接）则完全不同。它像是一张便签纸，上面写着“书房第二层书架从左数第三本”。这张便签纸本身不是书，只是一个指示位置的纸条。如果我按照纸条找到那本书，就可以阅读；但如果那本书被人拿走了或者扔掉了，我拿着这张纸条就什么也找不到，这就是“悬垂链接”。软链接和硬链接最大的区别在于：**硬链接是“多个名字指向同一个实体”，软链接是“一个名字指向另一个名字”。**

在文件系统中，硬链接的实现原理是在目录里增加一个新的目录项，这个目录项指向同一个inode（文件的身份证）。文件系统通过inode里的nlinks计数器记录有多少个目录项指向这个文件，只有当计数器减到0时才真正删除文件数据。软链接的实现则是创建一个特殊的文件，文件内容不是普通数据，而是目标文件的路径字符串。当我访问软链接时，操作系统会读取这个路径字符串，然后去查找真正的目标文件。

想要做出这个challenge，我们就要解决当多个进程同时创建或删除链接时，如何保证inode的引用计数nlinks正确无误；当进程A正在删除文件时，进程B同时要创建硬链接，如何防止竞态条件；还有软链接的路径解析可能需要多层查找，如何防止死循环（比如A链接指向B，B又指向A）。这些都需要通过合适的锁机制来解决。

以下是具体的设计实现：

1.核心数据结构设计

(1) 硬链接支持

硬链接实际上是在目录中添加一个新的目录项，指向同一个inode。现有的SFS数据结构基本支持硬链接，主要需要：


```

struct sfs_disk_inode {
    uint32_t size;                // 如果inode表示常规文件，则size是文件大小
    uint16_t type;                // inode的文件类型
    uint16_t nlinks;              // 此inode的硬链接数
    uint32_t blocks;              // 此inode的数据块数的个数
    uint32_t direct[SFS_NDIRECT]; // 此inode的直接数据块索引值（有SFS_NDIRECT个）
    uint32_t indirect;            // 此inode的一级间接数据块索引值
};

```

在这个完整的结构体中可以看到，`nlinks` 字段已经存在，位于 `type` 字段之后，用来记录有多少个目录项（硬链接）指向这个inode。当创建一个新的硬链接时，需要将这个值加1；当删除一个硬链接（比如通过`unlink`系统调用）时，需要将这个值减1。只有当 `nlinks` 减到0时，才表示没有任何目录项指向这个文件，此时可以安全地释放inode占用的磁盘空间和数据块。这个字段是实现硬链接机制的关键。

(2) 软链接（符号链接）支持

软链接是特殊的文件，内容为指向目标文件的路径：

```

// 软链接（符号链接）专用的inode扩展结构
struct symlink_inode {
    char *target_path;           // 目标路径字符串
    size_t path_len;             // 路径长度
    struct inode *target;         // 缓存的解析后目标inode（可选）
    uint32_t flags;              // 标志位
};

// inode中增加符号链接类型
struct inode {
    union {                      // 包含不同文件系统特定inode信息的union成员变量
        struct device __device_info; // 设备文件系统内存inode信息
        struct sfs_inode __sfs_inode_info; // SFS文件系统内存inode信息
        struct pipe __pipe_info;      // 管道信息
        struct symlink_inode __symlink_info; // 新增：符号链接信息
    } in_info;
    enum {
        inode_type_device_info = 0x1234,
        inode_type_sfs_inode_info,
        inode_type_pipe_info,          // 管道类型
        inode_type_symlink_info,       // 新增：符号链接类型
    } in_type;                         // 此inode所属文件系统类型
    atomic_t ref_count;                // 此inode的引用计数
    atomic_t open_count;               // 打开此inode对应文件的个数
    struct fs *in_fs;                  // 抽象的文件系统，包含访问文件系统的函数指针
    const struct inode_ops *in_ops;    // 抽象的inode操作，包含访问inode的函数指针
};

// 硬链接不需要额外的数据结构，使用现有的SFS结构
// 在sfs_disk_inode中已有nlinks字段
struct sfs_disk_inode {
    uint32_t size;                // 如果inode表示常规文件，则size是文件大小
    uint16_t type;                // inode的文件类型
    uint16_t nlinks;              // 此inode的硬链接数（已存在）
};

```



```

uint32_t blocks;                // 此inode的数据块数的个数
uint32_t direct[SFS_NDIRECT];   // 此inode的直接数据块索引值（有SFS_NDIRECT个）
uint32_t indirect;              // 此inode的一级间接数据块索引值
};

```

这段代码实现了软链接和硬链接的核心数据结构设计。**硬链接的实现比较简单，直接使用 `sfs_disk_inode` 结构体中已有的 `nlinks` 字段来计数有多少个目录项指向同一个文件，每当创建一个硬链接就把这个计数加一，删除一个链接就减一，计数减到0时才真正删除文件。**

软链接的实现则需要全新的数据结构，因为软链接本质上是一个特殊的文件，它的内容不是普通数据而是一个路径字符串。`struct symlink_inode` 就是为软链接设计的专门结构：`target_path` 存储目标文件的路径字符串，`path_len` 记录路径长度，`target` 是可选的缓存字段，用来存储已经解析好的目标inode指针，避免每次访问都重新解析路径。这个结构被放在 `struct inode` 的 `in_info` 联合体里，通过 `in_type` 字段的值 `inode_type_symlink_info` 来标识这是一个软链接类型的inode。

这样设计的好处是保持了文件系统接口的统一性——无论是普通文件、设备文件、管道还是软链接，都使用相同的 `struct inode` 结构，只是通过不同的 `in_type` 和 `in_info` 来区分具体类型。当用户访问一个软链接时，VFS会根据 `in_type` 调用专门为软链接设计的操作函数，这些函数会读取 `target_path` 的内容，然后解析并访问真正的目标文件。

2.关键接口设计

(1) 硬链接接口

```

// 创建硬链接：将新路径link_name链接到现有文件target_name
int hardlink(const char *target_name, const char *link_name);

```

这个 `hardlink` 函数的作用是**创建一个硬链接**，它接受两个参数：`target_name` 是已经存在的文件路径，`link_name` 是我们想要给这个文件起的另一个新名字（新路径）。函数执行成功后，系统中就会出现两个不同的路径名（`target_name` 和 `link_name`）都指向同一个实际的文件数据。就像给一个人取两个不同的绰号，无论我们用哪个绰号叫他，指的都是同一个人。从用户角度看，我们可以用任何一个名字来访问这个文件，进行读写操作都会影响到同一个文件内容。如果我们用 `link_name` 删除了文件，实际上只是删除了这个“绰号”，只要还有其他名字（比如原来的 `target_name`）指向这个文件，文件数据就还在。只有当所有名字（所有硬链接）都被删除时，文件数据才会真正被清除。

(2) 软链接接口

```

// 创建软链接：link_name指向target_path
int symlink(const char *target_path, const char *link_name);

// 读取软链接内容（不是跟随链接）
ssize_t readlink(const char *pathname, char *buf, size_t bufsize);

```

这两个函数是**软链接机制的核心接口**。`symlink` 函数用来创建一个软链接（符号链接），它接受两个参数：`target_path` 是目标文件的路径字符串，`link_name` 是要创建的软链接文件路径。调用这个函数后，系统会创建一个特殊的文件，这个文件的内容就是 `target_path` 这个字符串本身，而不是普通文件的数据。就像创建一张便利贴，上面写着“去书房第二层找《操作系统》这本书”，这张便利贴本身就是软链接文件，上面的文字就是目标路径。

`readlink` 函数的作用是读取软链接文件的实际内容（那个路径字符串），而不是去访问链接指向的目标文件。它接受三个参数：`pathname` 是软链接文件的路径，`buf` 是存放读取结果的缓冲区，`bufsize` 是缓冲区大小。调用这个函数后，它会读取软链接文件中存储的路径字符串，复制到 `buf` 里，返回实际复制的字节数。这就像我们看便利贴上写的文字是什么，而不是按照文字指示去找那本书。这个函数特别重要，因为有些系统工具（比如 `ls -l`）需要显示软链接指向哪里，还有些程序需要知道链接本身的信息而不是跟随链接去访问目标。

(3) 链接解析接口

```
// 解析链接（可能递归解析多层软链接）
struct inode *resolve_link(struct inode *link_inode, int depth);
```

这个 `resolve_link` 函数的作用是解析软链接并找到最终的目标文件。它接受两个参数：`link_inode` 是要解析的软链接文件的inode指针，`depth` 是最大递归深度（防止无限循环）。函数会读取软链接中存储的路径字符串，然后按照这个路径去查找目标文件。如果目标文件本身也是一个软链接，函数会继续解析，直到找到最终的非链接文件，或者达到最大递归深度。就像们拿到一张写着“去书房找红色笔记本”的便利贴（第一层软链接），我到书房找到红色笔记本，发现里面又有一张纸条写着“看第三个抽屉里的蓝色文件夹”（第二层软链接），我们再打开抽屉找到蓝色文件夹，这才看到真正的文件内容。这个函数会一层层地“剥开”软链接，直到找到最终的实际文件。`depth` 参数很重要，它能防止像“A指向B，B指向C，C又指回A”这样的循环链接导致函数永远无法结束。

3. 实现机制与同步处理

(1) 硬链接的实现机制

硬链接本质上是目录项的复制，核心是维护正确的引用计数：

```
// 创建硬链接的伪代码
int hardlink_create(const char *oldpath, const char *newpath) {
    // 1. 查找原文件的inode
    struct inode *old_inode = vfs_lookup(oldpath);
    if (!old_inode) return -ENOENT;

    // 2. 检查原文件是否为目录（通常不允许目录硬链接）
    if (S_ISDIR(old_inode->mode)) {
        vop_ref_dec(old_inode);
        return -EPERM;
    }

    // 3. 获取父目录inode和新文件名
    struct inode *parent_dir;
    char *basename;
    vfs_lookup_parent(newpath, &parent_dir, &basename);

    // 4. 在父目录中添加新目录项，指向同一个inode
    lock_inode(parent_dir);
    int ret = vop_create(parent_dir, basename, 0, NULL); // 不创建新inode
    if (ret == 0) {
        // 设置目录项指向原inode
        set_dirent_inode(parent_dir, basename, old_inode->ino);
        // 增加原inode的链接计数
        old_inode->nlinks++;
        old_inode->dirty = 1;
    }
}
```

```

unlock_inode(parent_dir);

// 5. 清理资源
vop_ref_dec(old_inode);
vop_ref_dec(parent_dir);
kfree(basename);
return ret;
}

```

这个函数是**创建硬链接的具体实现逻辑**。它的工作原理是这样的：首先根据 `oldpath` 找到原始文件的inode，确保文件存在并且不是目录（因为UNIX系统通常不允许给目录创建硬链接，否则可能产生循环目录结构问题）。然后解析 `newpath`，分离出父目录路径和新的文件名。接着锁定父目录，防止其他进程同时修改同一个目录。关键的一步是在父目录中添加一个新的目录项，但这个目录项不创建新的inode，而是指向原始文件的inode编号（`old_inode->ino`），这就像在通讯录里给同一个人添加第二个电话号码条目。然后增加原始inode的链接计数 `nlinks`，表示现在又多了一个名字指向这个文件。最后释放所有锁和资源。这样操作后，系统中就有了两个不同的路径指向同一个文件数据，无论通过哪个路径访问，操作的其实都是同一个文件，删除其中一个路径只是减少链接计数，只有当所有链接都被删除（链接计数减到0）时，文件数据才会真正被清除。

同步互斥处理：

- **目录项操作锁：**添加/删除目录项时需要锁住父目录。当进程A在某个目录下创建硬链接时，需要先锁住这个目录，确保进程B不能同时在这个目录里删除文件或创建其他链接，否则可能破坏目录结构的一致性。
- **inode引用计数原子操作：**文件被多少个名字引用的计数器操作必须不可分割。原子操作确保“读-改-写”这三个步骤一气呵成，不会被其他操作打断，这样计数器才能准确反映实际的链接数量。
- **删除文件时的处理：**只有当 `nlinks` 减为0时才真正释放inode和数据块。这个检查必须在持有适当锁的情况下进行，避免出现“A进程检查发现 `nlinks=1` 准备删除文件，同时B进程又创建了一个新硬链接把 `nlinks` 加到2”这种竞态条件，否则可能导致文件被误删或者链接计数错误。

(2) 软链接的实现机制

软链接是独立的文件，需要特殊处理：

```

// 软链接inode的创建
int symlink_create(struct inode *dir, const char *name, const char *target) {
    // 1. 创建普通文件，但标记为符号链接类型
    struct inode *link_inode;
    int ret = vop_create(dir, name, 0, &link_inode);
    if (ret != 0) return ret;

    // 2. 设置inode类型为符号链接
    link_inode->in_type = inode_type_symlink_info;

    // 3. 分配并设置符号链接信息
    struct symlink_inode *sinfo = &link_inode->in_info.__symlink_info;
    size_t target_len = strlen(target);
    sinfo->target_path = kmalloc(target_len + 1);
    strcpy(sinfo->target_path, target);
    sinfo->path_len = target_len;
    sinfo->target = NULL; // 延迟解析

    // 4. 设置文件大小为目标路径长度
    link_inode->size = target_len;
}

```

```
    return 0;
}
```

这个函数是**创建软链接文件的具体实现**。它的工作原理是在指定的目录 `dir` 下创建一个名为 `name` 的新文件，但这个文件不是普通的文件，而是一个特殊类型的文件——软链接。创建过程分四步：首先像创建普通文件一样调用 `vop_create` 创建文件并获取inode；然后把这个inode的类型标记为 `inode_type_symlink_info`，告诉系统这是个软链接而不是普通文件；接着为软链接分配专门的数据结构 `symlink_inode`，把目标路径 `target` 字符串复制到 `target_path` 字段里，就像在便利贴上写下“去某某地方找某某文件”；最后设置文件大小为路径字符串的长度。这样创建的软链接文件，当我们去读取它时，得到的是路径字符串本身，系统会根据这个字符串去查找真正的目标文件。`target` 字段设为 `NULL` 是延迟解析的优化，等真正需要访问目标文件时才去解析路径，避免不必要的开销。

链接解析的递归处理：

```
struct inode *resolve_symlink(struct inode *link_inode, int max_depth) {
    if (max_depth <= 0) return NULL; // 防止无限递归

    struct symlink_inode *sinfo = &link_inode->in_info.__symlink_info;

    // 如果已缓存解析结果，直接返回
    if (sinfo->target != NULL) {
        return sinfo->target;
    }

    // 解析目标路径
    struct inode *target = vfs_lookup(sinfo->target_path);

    // 如果目标也是符号链接，递归解析
    if (target && target->in_type == inode_type_symlink_info) {
        struct inode *resolved = resolve_symlink(target, max_depth - 1);
        vop_ref_dec(target);
        target = resolved;
    }

    // 缓存解析结果（注意引用计数）
    if (target) {
        sinfo->target = target;
        vop_ref_inc(target);
    }

    return target;
}
```

这个函数是**实现软链接解析的核心逻辑**，它的作用是“追根溯源”地找到软链接最终指向的实际文件。函数接受两个参数：`link_inode` 是软链接文件的inode指针，`max_depth` 是最大递归深度，用来防止无限循环的软链接（比如A指向B，B指向C，C又指回A）。

函数的工作原理是这样的：首先检查递归深度，如果已经达到极限就返回空，防止无限递归。然后检查这个软链接之前是否已经被解析过，如果 `symlink_inode` 结构里的 `target` 字段已经有缓存的结果，就直接返回缓存的目标inode，这是性能优化。如果没有缓存，就读取软链接文件中存储的路径字符串 `target_path`，调用 `vfs_lookup` 根据这个路径查找目标文件。关键的一步来了：如果找到的目标文件本身也是一个软链接，函数就递归调用自己，继续

解析下一层链接，但递归深度要减1，就像剥洋葱一样一层层剥开。最后，为了下次快速访问，函数会把解析结果缓存在 `target` 字段里，同时增加目标inode的引用计数，防止目标文件被删除后缓存指针变成悬垂指针。整个解析过程既考虑了效率（通过缓存），又保证了安全性（通过深度限制和引用计数管理）。

4.文件系统操作的特殊处理

(1) 打开文件时的链接处理

```
// 在vfs_lookup中需要处理符号链接
int vfs_lookup(const char *path, struct inode **node_store) {
    // ... 正常查找路径
    if (found_inode->in_type == inode_type_symlink_info) {
        // 如果是符号链接，解析它
        struct inode *target = resolve_symlink(found_inode, SYMLINK_MAX_DEPTH);
        vop_ref_dec(found_inode);
        if (!target) return -ELOOP;
        *node_store = target;
        return 0;
    }
    // ... 其他处理
}
```

这段代码展示了**软链接如何集成到虚拟文件系统的路径查找过程中**。当 `vfs_lookup` 函数在查找文件路径时，如果发现找到的文件inode类型是 `inode_type_symlink_info`（也就是软链接），它不会直接返回这个软链接的inode，而是调用 `resolve_symlink` 函数去解析这个链接，找到链接真正指向的目标文件。这就像我们按照地址找到一栋房子，发现门口贴着一张纸条写着“请去隔壁街5号”，我们不会认为这张纸条就是你要找的东西，而是会按照纸条上的指示继续去找真正的房子。

`resolve_symlink` 会递归解析软链接，最多解析 `SYMLINK_MAX_DEPTH` 层（通常是8层或16层），防止遇到循环链接时无限递归。如果解析成功，函数会减少软链接本身的引用计数（因为用户实际想要的是目标文件而不是链接本身），然后把目标文件的inode返回给调用者。如果解析失败（比如遇到循环链接或超出最大深度），就返回 `-ELOOP` 错误码。这样设计确保了当用户打开一个软链接文件时，实际上打开的是链接指向的目标文件，符合UNIX系统的预期行为。

(2) 删除文件时的处理

硬链接删除：

```
// 删除文件（硬链接）
int unlink(const char *pathname) {
    // 1. 查找文件的inode和父目录
    // 2. 从父目录中移除目录项
    // 3. 减少inode的nlinks计数
    // 4. 如果nlinks变为0，释放inode和数据块
    // 5. 如果nlinks > 0，只删除目录项
}
```

这个 `unlink` 函数是**实现硬链接删除操作**的核心逻辑。它的作用是删除一个文件路径名（硬链接），但不同于普通删除，它只删除这个“名字”而不一定删除文件数据。函数的工作流程是这样的：首先根据 `pathname` 找到要删除的文件及其所在的父目录；然后从父目录中移除对应的目录项，就像从通讯录里划掉一个人的一个电话号码；接着减少文件inode的链接计数 `nlinks`，表示指向这个文件的“名字”少了一个。关键决策在第四步：如果 `nlinks` 减到0，说明这

是指向该文件的最后一个名字了，就像一本书的所有借阅记录都被清空了，这时候需要真正释放文件占用的inode和数据块空间，文件就彻底消失了；如果 `nlinks` 还大于0，说明还有其他硬链接指向这个文件，那么只删除当前这个目录项即可，文件数据继续保留。这个机制实现了文件的共享和资源管理，多个路径可以共享同一份文件数据，只有当所有路径都被删除时文件才真正被清除。

软链接删除：

```
// 删除软链接（与删除普通文件相同，但只删除链接文件本身）
int unlink_symlink(const char *pathname) {
    // 与普通文件删除类似，但需要注意：
    // 1. 软链接的删除不影响目标文件
    // 2. 需要释放软链接的特殊数据结构（target_path）
}
```

这个 `unlink_symlink` 函数是**用来删除软链接文件的**。虽然函数名叫“删除软链接”，但它的行为其实和删除普通文件很相似，因为软链接本身就是一个独立的文件，只不过这个文件的内容是一个路径字符串而不是普通数据。函数执行时会先找到软链接文件的inode，然后像删除普通文件一样从父目录中移除对应的目录项，释放inode占用的资源。

但有两个关键区别需要注意：**第一，删除软链接完全不影响它指向的目标文件**，就像撕掉一张写着“书房第二层有《操作系统》”的便利贴，并不会对书房里的书造成任何影响，书还在原处。**第二，由于软链接有自己特殊的数据结构 `symlink_inode`，里面存储着 `target_path` 字符串，所以在释放inode之前需要先释放这个字符串占用的内存，避免内存泄漏**。这就像不仅要扔掉便利贴，还要把上面写的字迹也清理干净。删除后，目标文件依然存在且可以正常访问，只是少了一个指向它的“快捷方式”而已。

5.同步互斥的详细方案

(1) 硬链接的竞态条件处理

- **目录项操作**：使用父目录的inode锁保护目录项修改。**目录项操作**需要加锁保护是因为目录本质上是一个文件，里面记录了文件名和inode号的对应关系。当进程A在某个目录下创建硬链接时，它实际上是在修改这个目录文件的内容——增加一个新的目录项。如果不加锁，进程B可能同时删除同一个目录下的另一个文件，两个进程同时修改同一个目录文件可能导致目录结构损坏，比如出现重复的目录项或者丢失某些条目。这就好比两个图书管理员同时修改同一本图书目录，一个人正在添加新书条目，另一个人正在删除旧书条目，如果不协调好，目录就可能出现混乱。
- **引用计数更新**：使用原子操作或锁保护 `nlinks` 字段。**引用计数更新**需要原子操作是因为 `nlinks` 字段记录着有多少个目录项指向同一个文件。假设文件当前有2个硬链接，进程A和进程B同时执行删除操作（`unlink`）。如果不使用原子操作，可能出现这样的竞态：A读取 `nlinks=2`，准备减1；B也读取 `nlinks=2`，准备减1；A减1后写回1；B减1后也写回1。但实际上两个进程都执行了删除，`nlinks` 应该变为0，文件应该被真正删除。原子操作确保“读取-修改-写入”这个过程不可分割，就像只有一个管理员在操作借阅计数器，一次只处理一个借还书请求。
- **删除操作**：检查 `nlinks` 和删除目录项需要原子操作。**删除操作**的原子性要求更严格。当进程准备删除最后一个硬链接时，它需要做两件事：检查 `nlinks` 是否等于1，如果等于1就真正删除文件。但如果这个过程不是原子的，可能出现这样的危险情况：进程A检查发现 `nlinks=1`，准备删除文件；与此同时，进程B正在创建新的硬链接，把 `nlinks` 加到2；然后A继续执行删除了文件，但此时文件实际上还有B创建的链接存在。这就像图书管理员看到某本书的借阅记录只剩一条，准备下架这本书，但就在他下架的过程中，另一个读者又借走了这本书。原子操作确保“检查计数-执行删除”这两个步骤要么全部完成，要么都不做，避免这种不一致状态。

(2) 软链接的缓存一致性

- **链接解析缓存**：使用读写锁保护target缓存字段。**链接解析缓存**需要读写锁是因为软链接解析可能比较耗时（要读磁盘找路径再找目标），所以把解析结果缓存在 `target` 字段里。但多个进程可能同时访问同一个软链接，如果都用写锁会降低并发性，读写锁允许多个读进程同时访问缓存结果（比如多个进程都打开同一个软链接文件），但写进程（更新缓存时）需要独占访问。
- **目标文件删除**：需要处理悬垂指针（可设置失效标志）。**目标文件删除**会导致悬垂指针问题：软链接缓存了目标文件的inode指针，但如果目标文件被删除了，这个指针就指向了无效内存。可以设置失效标志，当发现目标文件被删除时标记缓存失效，下次访问时重新解析（如果目标文件又恢复了就解析新的，否则报错）。
- **路径解析**：多层软链接解析需要限制深度，防止死循环。**路径解析**需要限制深度是因为软链接可能形成循环链：A指向B，B指向C，C又指回A。如果不限解析深度，解析函数会无限递归下去直到栈溢出。通常设置最大深度为8或16层，超过就报"ELOOP"错误，这就像防止你按照"去A房间看B纸条，B纸条说去C房间，C纸条又说回A房间"这样的无限循环指示。

(3) 与现有VFS的集成点

- **路径解析**：修改`vfs_lookup`支持符号链接跟随。**路径解析**集成需要在 `vfs_lookup` 函数中增加软链接处理逻辑。当查找路径时遇到软链接类型的inode，不能直接返回这个链接的inode，而要递归解析它指向的目标路径。就像按照地址找到一栋房子，发现门口贴着一张"请去隔壁街"的纸条，你得按纸条指示继续找，直到找到真正的房子。但这里要有限制，防止纸条链成循环（A指向B，B指向C，C又指回A）。
- **文件操作**：某些操作（如`readlink`）需要特殊处理符号链接。**文件操作**的特殊处理体现在有些操作要针对链接本身而非目标。比如 `readlink` 系统调用就是要读取软链接文件中存储的路径字符串，而不是去读目标文件的内容。这就像你想知道纸条上写的什么地址，而不是按地址去找房子。而像 `open`、`read`、`write` 这些操作默认应该跟随链接，操作目标文件。
- **权限检查**：软链接的权限检查应针对链接文件本身，而非目标文件。**权限检查**的规则是：对软链接本身的元数据操作（比如查看链接文件的属性、修改链接文件的权限）检查链接文件的权限；但对链接内容的操作（比如读取链接指向的文件）检查目标文件的权限。就像一张便利贴本身的归属和内容（地址信息）归贴的人管，但按地址找到的房子归房主管，你要进房子得房主同意，但看便利贴上的字只要贴的人允许就行。